MAGNIT TECH

Лекция 6

S3 как основа современного Data Lake. Row-based и columnar форматы. Партиционирование и компрессия.

Курс Data Engineering | MAGNIT TECH x НИУ ВШЭ

План лекции

Hadoop и HDFS: с чего всё начиналось

S3: бакеты, объекты, версии, жизненный цикл

Строковые (row-based) форматы: CSV, JSON, Avro

Колоночные (columnar) файловые форматы: ORC, Parquet

Табличные форматы: Apache Iceberg, Delta Lake

Row-based vs columnar: что выбираем

Партиционирование и компрессия: производительность и стоимость

MAGNIT TECH

Hadoop и HDFS:
с чего всё начиналось

Немного истории

2000-е: объёмы данных растут → в одну серверную стойку не помещаются, специализированные массивы стоят дорого

Google (статьи про GFS и MapReduce): не собирай данные на одном мощном сервере → распредели данные и задачи на множестве недорогих серверов и обрабатывай параллельно

Hadoop

Открытой реализацией идей от Google стал **Hadoop**

Hadoop – свободно распространяемый набор утилит, библиотек и фреймворк для разработки и выполнения распределённых вычислений



Состав Hadoop

HDFS (*Hadoop Distributed File System*) – распределённая файловая система: разбивка файлов на блоки, каждый блок хранится с репликами на нескольких машинах

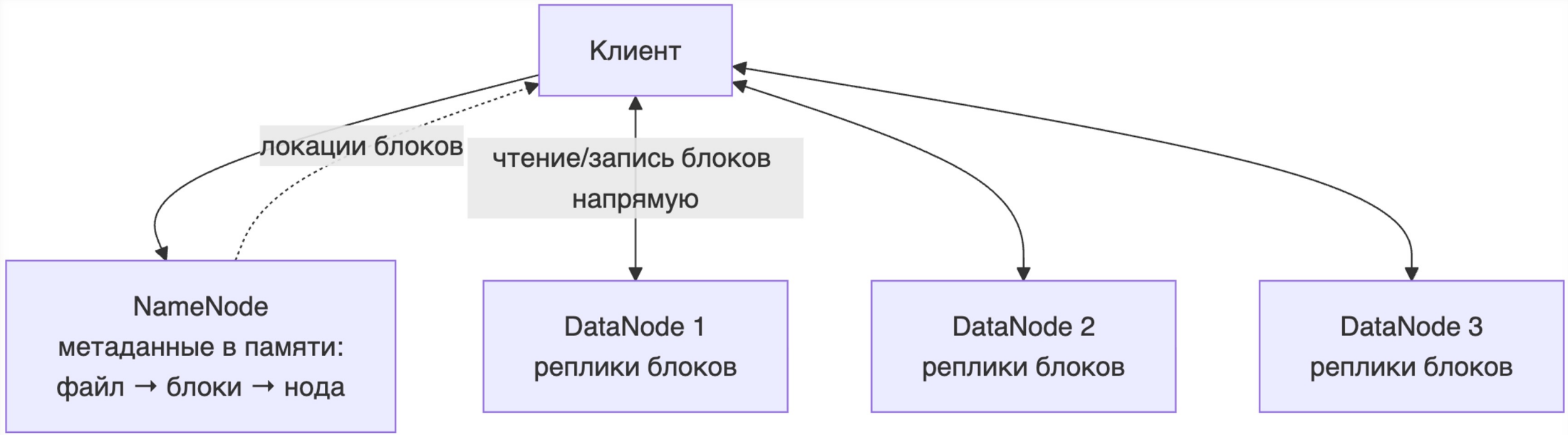
- **NameNode** – «каталог»: помнит, где какой блок лежит
- **DataNode** – обычные серверы с дисками

YARN (*Yet Another Resource Negotiator*) – система управления ресурсами кластера и планирования задач

MapReduce – модель вычислений, код выполняется **рядом с данными**

Hadoop Common – общие библиотеки и утилиты, поддерживающие другие модули

Архитектура HDFS



Механика:

- *клиент спрашивает у NameNode, где лежат блоки нужного файла*
- *читает их напрямую с DataNode, выбирая ближайшую реплику*
- *запись идет «конвейером» через несколько DataNode подряд, так по пути создаются реплики*

❓ Какие у подобной архитектуры могут возникнуть проблемы?

Проблемы HDFS

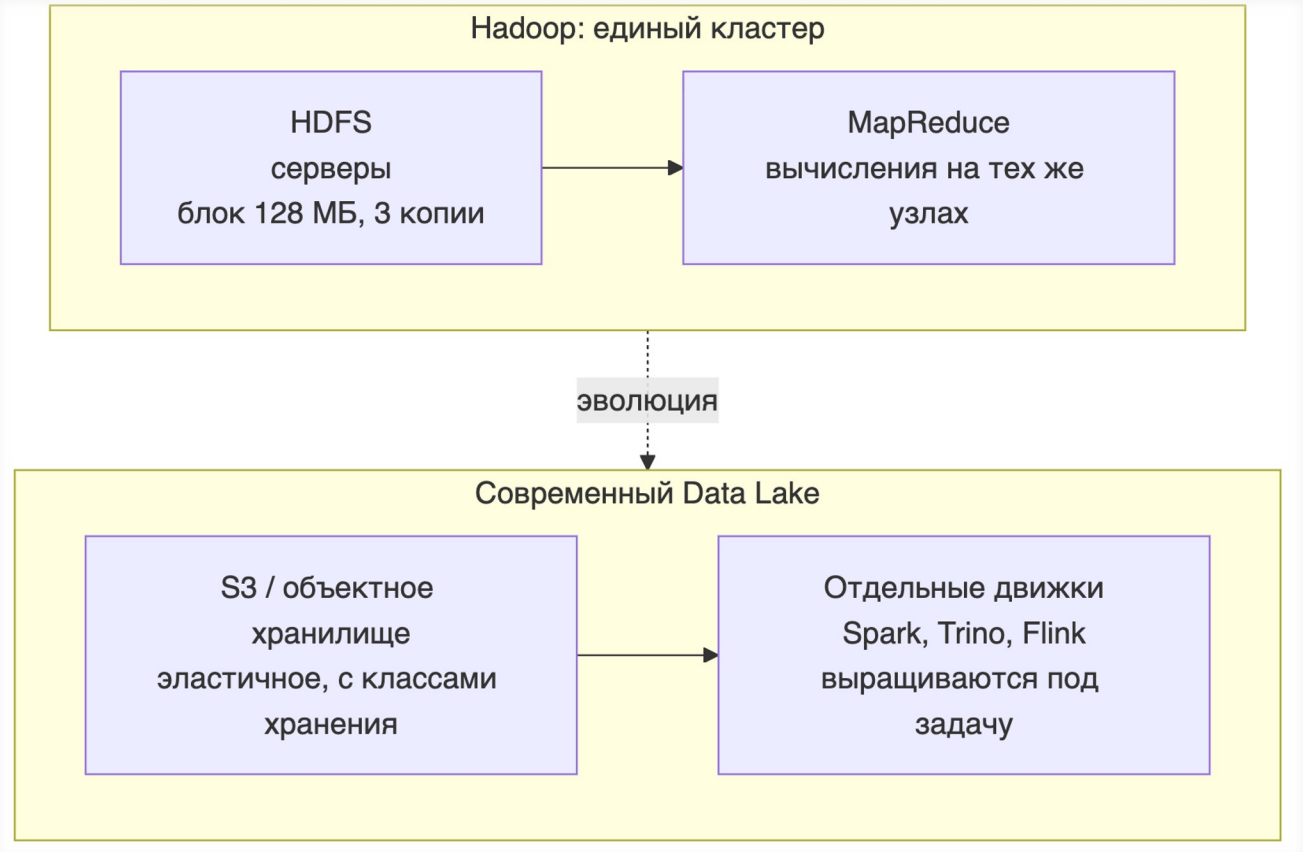
Хранение привязано к железу: удвоить хранилище = добавить машины (хотя нужны они в разные моменты)

Хранение и расчёт не разделены.

NameNode: единая точка метаданных, масштаб упирается в оперативную память ноды

Высокая стоимость: создание и расширение кластера требует больших вложений

Эволюция от Hadoop-кластера к S3-озеру



***Важно**! Hadoop до сих пор используется в многих коммерческих проектах*

MAGNIT TECH

S3 как основа Data Lake

Что уже знаем про Data Lake

Data Lake – хранилище больших объёмов данных в исходном или слабо обработанном виде, чаще всего поверх объектного хранилища, совместимого с **S3**, в которое загружают CSV, JSON, Parquet, логи, дампы, картинки

Но что же такое S3?

S3

S3 (Simple Storage Service) – это стандарт объектных хранилищ, созданный компанией Amazon Web Services (AWS) в 2006 году, в тот же год, что и Hadoop.

Критерий	S3	HDFS
Модель	объектное хранилище: бакет → ключ → объект	распределённая ФС: блоки, пути
Масштабирование	отделено от вычислений, платишь за место	привязано к кластеру
Изменение данных	объект неизменяем, только перезапись	файл на дисках кластера
Порог входа	API, без своего кластера	свой кластер серверов

Модель S3: бакеты, ключи, объекты

В S3 есть три сущности:

Бакет – глобально уникальное «корневое имя» пространства объектов. В облаке на S3 имя бакета уникально в рамках региона или проекта. Также бакет – это контейнер с настройками: права, классы хранения, lifecycle, логирование.

Объект – то, что реально хранится: байты + метаданные + ключ(key). Объект в S3 ***нельзя изменить***, его можно ***только перезаписать***.

Ключ – строка, которая выглядит как путь:

s3://de-course/**raw**/orders/load_date=2026-09-01/orders.parquet

└ бакет ┘ └ префиксы («папка») ┘ └ имя файла ┘

Структура сырого слоя Data Lake

Структура сырого слоя Data Lake:

s3://de-course/
    raw/
        orders/
            load_date=2026-08-31/0000.parquet
            load_date=2026-09-01/0000.parquet
        events/
            day_id=2026-09-01/0000.parquet
        products/…

Фактическое расположение объектов:

s3://de-course/**raw**/orders/load_date=2026-08-31/0000.parquet
s3://de-course/**raw**/orders/load_date=2026-09-01/0000.parquet
s3://de-course/**raw**/events/day_id=2026-09-01/0000.parquet
s3://de-course/**raw**/events/products/…



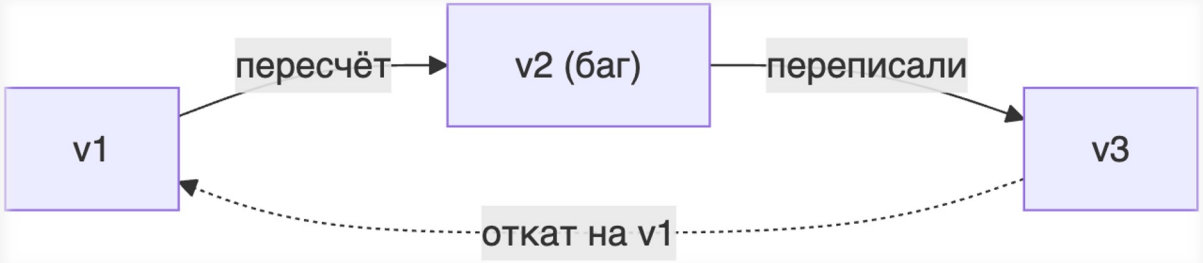
Важно! Название слоя – это часть префикса ключа

Версионирование объектов S3

Versioning – опция на бакете, каждая запись объекта с тем же ключом создаёт новую версию вместо удаления старой

Зачем это в Data Lake:

Защита от человека. Пайплайн перезапустился и перезаписал данные за день – старая версия на месте.



Аудит и compliance. Кто, когда, что записал – история хранится.

Безопасная перезапись. Некоторые компоненты (Spark, инструменты миграции) при включённом версионировании становятся безопаснее: «удаление» превращается в mark-for-deletion.

Цена версионирования объектов S3

Каждая версия хранится и **оплачивается**

Мусор от неудачных пайплайнов будет копиться

Правило: версионирование почти всегда включают вместе с **lifecycle policy** «неактуальные версии старше N дней – удалить»

Жизненный цикл данных (lifecycle)

Lifecycle policy – правила на бакете: по прошествии N дней объект автоматически:

переходит в более дешёвый класс хранения (Стандартное → Холодное → Ледяное)

или удаляется

Классы хранения – это компромисс «цена хранения – время доступа»:

Класс хранения	Доступ к данным	Стоимость	
		Запросы	Хранение данных
Стандартное (Standard)	Часто	Низкая	Высокая
Холодное (Standard-IA)	Редко, примерно раз в месяц	Средняя	Средняя
Ледяное (Glacier)	Очень редко, примерно раз в год или реже	Высокая	Низкая

❓ Пайплайн каждую ночь обновляет данные за предыдущую неделю (перезаписывает папки `raw/orders/load_date=****-**-**/` новыми файлами), старые объекты остаётся лежать. Что будет через полгода? Что бы вы включили?

MAGNIT TECH∞

Строковые (row-based) форматы:
CSV, JSON, Avro

Общий принцип row-based форматов

Строка целиком хранится подряд: все значения одной записи лежат рядом

Удобно читать/писать целую запись

Неудобно для аналитики: нужны 2 колонки из 50 – читаем всё

CSV

CSV (Comma-Separated Values) – это простой текстовый формат данных

Особенности:

значения через разделитель

заголовок (название колонок) есть, но может и отсутствовать

Фрагмент CSV:

```
order_id,customer_id,order_created_at,status,total_amount
9001,42,2026-09-01T10:00:00,paid,4990.00
9002,57,2026-09-01T10:03:12,new,1250.50
9003,42,2026-09-01T10:07:41,cancelled,899.90
```

CSV

Плюсы:

Универсальность: читает и пишет буквально каждый инструмент

Человекочитаемость: можно открыть в редакторе и за 10 секунд понять, что случилось

Простота: нет метаданных, нет структуры – только текст

Минусы:

Нет типов: `4990.00` — число или строка? `2026-09-01T10:00:00` – таймзона?

Разделители: запятая внутри значения, кавычки, перенос строк в тексте – классика багов парсинга

Схема не в файле: что за 7-я колонка – узнаёшь в документации или коллеги

Размер: текст сжимается заметно хуже бинарных колоночных форматов

Нет механизма «пропусти мне ненужные части»: При чтение требуется распарсить каждую строку целиком

Когда CSV правильный выбор:

Выгрузка/импорт между системами, где формат – договорённость

Raw-слой небольшого статичного справочника

Быстрый ad hoc: «даны данные, дай результат через час»

JSON

JSON (JavaScript Object Notation) – текстовый формат для хранения и передачи структурированных данных

Особенности:

- **Объекты ({})**: неупорядоченный набор пар «ключ: значение»
- **Массивы ([])**: упорядоченный список значений
- **Поддерживаемые типы данных**: строки, числа, логические значения, null, объекты и массивы

Фрагмент JSON:

```
{"order_id": 9001, "customer_id": 42, "amount": 4990.0, "currency": "RUB", "items": 3}
{"order_id": 9002, "customer_id": 57, "amount": 1250.5, "currency": "RUB", "items": 1, "promo_code": "X5"}
```

JSON

Плюсы:

Человекочитаемость

Универсальность и кроссплатформенность

Нативная вложенная структура

Минусы:

Ключи повторяются в каждой записи: в CSV шапка одна на файл, в JSON – имена полей на каждую строку. Это десятки процентов «мусорного» объема

Размер: при равных данных JSON обычно в 2–5 раз больше, чем тот же набор в Parquet.

Где JSON уместен в Data Lake:

сырые события (логи, API-выгрузки)

«принять как есть» с переработкой дальше

Avro

Avro – бинарный формат хранения и передачи структурированных данных

Особенности:

данные сериализуются в компактный двоичный формат

структура данных описывается с помощью JSON

Avro

Плюсы:

Схема встроена в файл: файл сам знает, что такое `order_id` и какого он типа

Бинарный и компактный: файл сам знает, что такое `order_id` и какого он типа

Эволюция схемы по правилам: поле с default – старые файлы читаются; убрали поле — лишнее игнорируется

Минусы:

Нечеловекочитаемый: «взгляд глазами» уже не тот, что у CSV

Row-based: данные лежат построчно, для чтения «двух колонок из миллиона строк» не оптимален

Место Avro в Data Lake:

в потоковых системах (Kafka) Avro + Schema Registry – стандарт, отправители и получатели эволюционируют независимо

Сравнение row-based форматов

Критерий	CSV	JSON	Avro
Человекочитаемость	Высокая	Средняя	Низкая
Типы	Нет	Частично (в значениях)	Да, в схеме
Схема	Нет (только шапка)	Поля в записи	В файле
Компактность	Средняя	Низкая (избыточные имена)	Высокая
Вложенность	Плохо	Хорошо	Хорошо
Эволюция схемы	Нет	Гибко, неформально	По правилам
Чтение части колонок	Нет	Нет	Нет
Типовое место	Обмен/интеграции	Raw-события	Потоки (Kafka)

MAGNIT TECH

Колоночные (columnar) файловые форматы: ORC, Parquet

В прошлых сериях

Почему колоночное хранение помогает аналитике:

- читаются только нужные столбцы
- одинаковые типы и похожие значения хорошо сжимаются
- значения обрабатываются блоками – удобно для CPU
- метаданные и индексы позволяют пропускать целые куски

Parquet и ORC – те же принципы, но упакованные в файл

Разница с ClickHouse – в масштабе: файл кладём в S3, читает любой движок

ORC

ORC – это колоночный файловый формат, **исторический** стандарт таблиц Hive (Hadoop-мир)

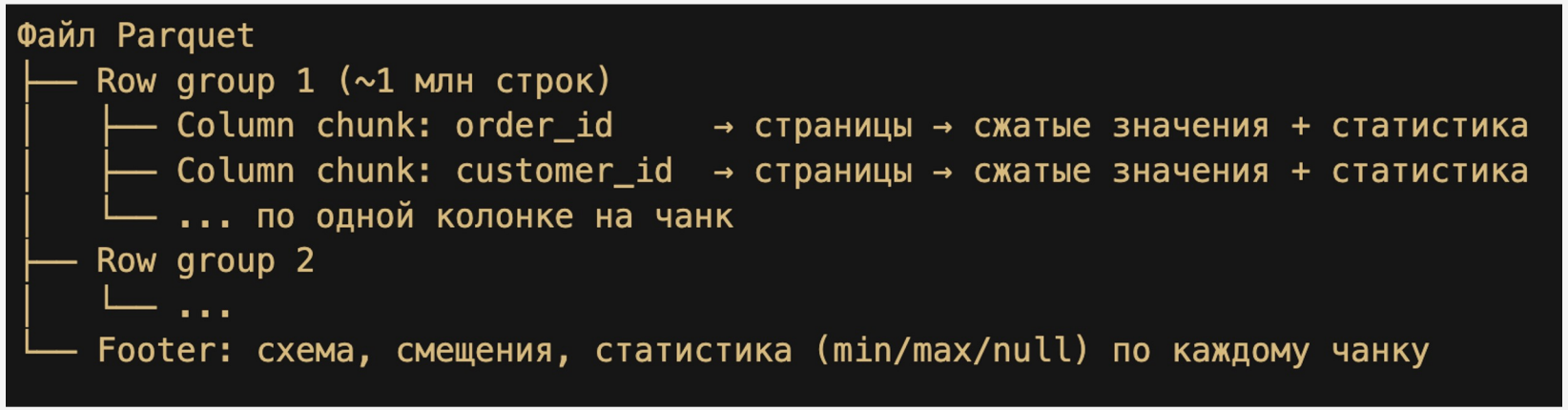
Особенности:

- Данные разбиты на **stripes** (по умолчанию ~256 МБ) – единицу параллельного чтения
- Внутри stripe – колонки, сжатые независимо, со своей статистикой (min/max, количество null)
- Статистика позволяет движку **пропускать** целые куски, не читая их
- «Встроенные» опции: bloom-фильтры для точечных фильтров, ACID-режим в Hive

Parquet

Parquet – это колоночный файловый формат, который **де-факто** является **стандартом** колоночных файлов в **Data Lake**

Структура файла Parquet



Файл состоит из **row group**

Данные внутри **row group** лежат по колонкам

Каждый **column chunk** сжимается своим кодеком

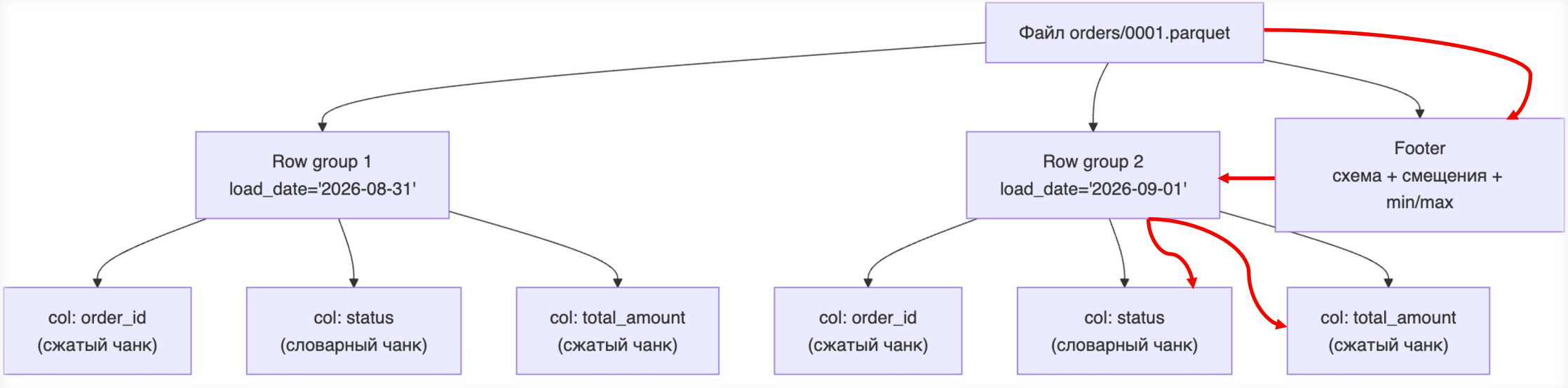
Footer хранит схему, размеры, смещения и статистики min/max/null для каждого чанка

Dictionary encoding для колонок с низкой кардинальностью – словарь, а не повторение строк

Как происходит чтение данных Parquet

Пример: запрос «выручка за 1 сентября по статусу paid» при чтении данных в Parquet:

1. читается **footer** (он небольшой) – движок узнает схему и статистику, **не трогая данные**
2. по min/max даты отбрасывает **row groups**, где 1 сентября нет
3. читает **только чанки** `status` и `total_amount`, а не весь файл
4. внутри чанков работа со сжатыми блоками



Минусы Parquet

Нельзя «поменять одну строку»: меняем, перезаписывая файлы

Запись дороже: небольшие изменения требуют ресурсов на запись всех данных

Собрать одну широкую строку по ключу: собрать значения из многих колонок

Parquet vs ORC

Оба – columnar, сжимают по колонкам, несут статистики

Parquet – де-факто стандарт lake-экосистемы (Spark, Trino, Presto, DuckDB и пр.)

ORC – наследник Hadoop/Hive-миров, чуть другая упаковка (stripes), популярен там, где исторически используется Hive

Вывод: *новый Data Lake → Parquet, живущий Hive-мир → ORC*

MAGNIT TECH

Табличные форматы:
Apache Iceberg, Delta Lake

В прошлых сериях

Папка с файлами – это еще не таблица 

Список вопросов, на которые «папка» не отвечает:

- какие файлы сейчас входят в таблицу
- как атомарно заменить часть данных
- что увидит читатель во время перезаписи
- как удалить или обновить строки
- как вернуться к предыдущей версии
- как безопасно менять схему
- как несколько процессов пишут одновременно

Решение:

Табличные форматы данных – данные в Parquet + дерево метаданных, которое описывает, что сейчас считается таблицей

Apache Iceberg

Apache Iceberg – табличный формат с «классической» системой метаданных, устроенной как дерево:

```
catalog (Hive / Nessie / REST)
   └── metadata v42.json        – версия таблицы: схема, список snapshot'ов
         └── snapshot-123
               └── manifest-list-7
                     ├── manifest-1.json  → файлы: data/load_date=2026-09-01/part-0001.parquet, …
                     ├── manifest-2.json  → файлы: data/load_date=2026-09-01/part-0002.parquet, …
                     └── …
```

Ключевые свойства Apache Iceberg

ACID* на уровне таблицы. Коммит – это смена ссылки на snapshot. Читатель видит либо старую версию таблицы, либо новую. «Половины дня» не бывает. Параллельные коммиты не «съедаются»: при конфликте один из коммиттеров пересобирает коммит поверх актуального состояния

Time travel. Старые snapshot'ы не удаляются мгновенно – можно прочитать таблицу «как три дня назад»

Hidden partitioning. Партиционирование описано в метаданных, а не «зашито» в именах папок – его можно поменять, не переливая данные

Безопасная эволюция схемы. Добавили колонку – старые файлы читаются, в них новой колонки просто нет (NULL)

Независимость от движка. Spark, Trino, Flink читают одну таблицу по одним правилам

** ACID (atomicity, consistency, isolation, durability) – набор требований к транзакционной системе, обеспечивающий наиболее надёжную и предсказуемую её работу – атомарность, согласованность, изоляцию, устойчивость*

Delta Lake

Delta – конкурент с другим акцентом, метаданные в виде **журнала транзакций** `_delta_log/` рядом с данными:

```
data/
  load_date=2026-09-01/
    part-0000.parquet
_delta_log/
  00000000000000000000.json   ← "добавил файл X"
  00000000000000000001.json   ← "удалил файл Y"
  00000000000000000002.checkpoint.parquet
```

Ключевые свойства Delta Lake

ACID через лог транзакций. Каждая операция (ADD/DELETE файлов) – запись в логе; читатель применяет лог и получает состояние на версию

DML* прямо на данных. В Iceberg тоже есть, но Delta исторически «DML-ориентированнее»

Time travel по версии/времени. Аналогично Iceberg

Зрелая интеграция со Spark

** **DML** (Data Manipulation Language) – это язык манипулирования данными, подмножество языка SQL, основные команды INSERT, UPDATE, DELETE, MERGE*

Apache Iceberg vs Delta Lake

Критерий	Apache Iceberg	Delta Lake
Данные	Parquet (и другие)	Parquet
Механика состояния	Snapshot → manifest list → manifests	Транзакционный лог `_delta_log`
Time travel	По snapshot	По версии лога
Смена партиционирования без перезаписи	Да (hidden partitioning)	Ограниченно
Нейтральность к движкам	Высокая	Высокая (с уклоном в Databricks)
Типовое место	Открытые multi-engine платформы	Платформы вокруг Databricks

MAGNIT TECH

Row-based vs columnar: что выбираем

Row-based vs columnar

Критерий	Row-based	Columnar
Выигрывает, когда	читать/менять целую строку (OLTP)	агрегировать много строк по немногим колонкам (OLAP)
Форматы	CSV, JSON, Avro	Parquet, ORC
Место в lake	raw-слой, интеграции, потоки	raw-слой, базовый и аналитический слой

MAGNIT TECH

Партиционирование и компрессия: производительность и стоимость

Партиционирование

Партиционирование – разделение больших таблиц на на более мелкие логические части

В контексте хранения в S3 – разбиение данных таблицы по дополнительным вложенным каталогам ключ=значение (префиксам ключей)

Партиционирование позволяет:

ускорить выполнение запросов – чтение только нужной партиции

оптимизировать управление данными – перезапись партиций

масштабировать таблицу – добавление новой партиции

Таблица с партиционированием и без

```
SELECT * FROM orders WHERE load_date = '2026-09-01'
```

БЕЗ партиционирования:

```
s3://de-course/raw/orders/
```

→ *много файлов → читаем ВСЕ файлы, фильтруем по содержимому*

С партиционированием:

```
s3://de-course/raw/orders/
    load_date=2026-08-31/
        part-0000.parquet
        part-0001.parquet
    load_date=2026-09-01/
        part-0000.parquet
        part-0001.parquet
```

→ *1 папка, пара файлов → движок смотрит префикс и читает только эту папку*

Правила хорошего партиционирования

Под запросы, а не под «логику схемы»: по каким полям почти все запросы фильтруют?»

Низкая кардинальность (количество уникальные значения):

`load_date` (365) – хорошо

`customer_id` (миллионы) – плохо (миллионы каталогов, медленный list, небольшие файлы)

Каскад неглубокий. 1–2 уровня (`load_date=…/country=…`)

Hive-стиль именования `key=value`: стандарт, который понимают Spark, Trino, Iceberg, dbt

Коллега предлагает партиционировать таблицу заказов по `status`: `status=paid/`, `status=new/`. Что думаете об этом?

Компрессия

Компрессия – уменьшение объема данных за счет удаления избыточности и более компактного кодирования данных

Два уровня:

- **Кодек (snappy, zstd, lz4, gzip):** как именно сжимаются байты внутри файла
- **Формат (Parquet/ORC):** компрессия **внутри колонок**, где значения однородные (эффективно именно поэтому)

Кодеки: скорость против плотности

Кодек	Скорость сжатия	Степень сжатия	Когда
gzip	Медленнее	Высокая	Оффлайн-архивы, где скорость не критична
snappy	Очень быстро	Средняя	Исторический выбор Hadoop/Spark
lz4	Очень быстро	Средняя	Когда важна скорость чтения
zstd	Быстро	Высокая	**Золотая середина, типовой современный выбор**

Кодек выбирается при сохранении файлов Parquet

Эффект на реальных данных

Состояние raw-слоя	Объем (пример: 1 ТБ исходных событий)	Чтение «одного дня»	S3-запросы
JSON, без партиционирования, мелкие файлы	~1000 ГБ	Читаем все, фильтруем по содержимому	Много (мелкие GET)
Parquet + zstd, партиции по дню	~100–200 ГБ	Читаем одну папку + skipping по row groups	Мало (крупные GET)

Вывод

Производительность и **стоимость** Data Lake определяются не движком и не кластером, а физической организацией данных:

- формат данных
- степень сжатия данных
- партиционирование
- lifecycle
- размер файлов

Несколько тезисов в завершение

Hadoop/HDFS – корень: большие блоки, реплики, метаданные отдельно от данных, «мелкие файлы – враг».

S3 – не файловая система: бакет → ключ → объект; объект неизменяем; ключ – это архитектура.

Версионирование и lifecycle – откат от багов и лестница стоимости (Standard → IA → Glacier).

CSV / JSON / Avro – row-based форматы: честные, живучие, но для аналитики не оптимум.

ORC, Parquet, Iceberg, Delta Lake – от колоночного формата до табличных слоев с транзакциями.

Row vs columnar – читаем нужное, сжимаем сильно, пропускаем ненужное.

Партиционирование и компрессия – то, что превращает «хранилище» в «дешевое и быстрое хранилище».

Вопросы



MAGNIT TECH

MAGNIT TECH

Спасибо за внимание!

